

SOUTHEAST UNIVERSITY
Department of Computer Science and Engineering (CSE)

Assignment 1
Java Collections & I/O with REST API

Course Title: Advanced Java Lab
Course Code: CSE352
Section: 4

Submitted To:
Miftahul Sheikh
Faculty, Department of CSE
Southeast University

Submitted By:
Name: Ahnaf Sakil Mahmud
ID: 2022100010066
Program: BSc in CSE - Batch 14

Date of Submission: October 1, 2026

Objective

The primary objective of this assignment is to develop a Java-based RESTful Student Management System. The project practically demonstrates the implementation of the Java Collection Framework (`ArrayList`, `HashSet`, `Stack`), sorting mechanisms (`Comparable` and `Comparator`), Iterators, and Java I/O using Object Serialization.

Implementation Summary

The project was built using the Spring Boot framework to expose RESTful API endpoints, fulfilling all core requirements:

- `ArrayList`: Utilized as the primary in-memory data structure to store `Student` objects.
- `Comparable` & `Comparator`: The `Student` class implements `Comparable` to sort students by ID ascending. A separate `StudentCgpaComparator` was implemented to sort students by CGPA in descending order.
- `HashSet`: Utilized to extract and return a collection of unique student departments, automatically discarding duplicates.
- `Stack`: Used to maintain a history of the last three added students, successfully demonstrating the use of `push()`, `pop()`, and `peek()` operations.
- `Iterator`: Used explicitly within the service layer to safely traverse and return the list of all students.

Java I/O: Byte Streams vs. Character Streams

The application uses Java I/O Object Serialization to save the student list to a file and restore it upon application startup. Below is a comparison between the two primary types of Java I/O streams:

Byte Streams

Byte Streams process data exactly one byte (8 bits) at a time. They are strictly used for handling raw binary data, such as images, audio files, or serialized Java objects.

- **Usage in this Project:** This project relies on Byte Streams. Specifically, `FileOutputStream` and `ObjectOutputStream` are used to serialize the `ArrayList` of students into a binary `.ser` file. Upon restarting the application, `FileInputStream` and `ObjectInputStream` deserialize the binary data back into living Java objects.
- **Examples:** `FileInputStream`, `FileOutputStream`, `BufferedInputStream`.

Character Streams

Character Streams process data character by character (16-bit Unicode). They are optimized specifically for handling plain text data and automatically translate data to and from the local character set format.

- Use Case: Character streams should be used when reading or writing human-readable text files (e.g., `.txt`, `.csv`, or `.json` files). If we had saved the student data as a comma-separated text file instead of a serialized object, we would have used Character Streams.
- Examples: `FileReader`, `FileWriter`, `BufferedReader`.